Probabilistic analysis and randomized algorithms

Part I Foundations

Scope and prerequisites

Fix the sample space and dependence assumptions first. Hiring indicators and a general uniform-shuffle proof lead to birthday collisions, bin waiting times, overlapping streaks, and online selection. Random recurrences and Quicksort distinguish expectations, event probabilities, worst cases, and tails. Prerequisites are discrete probability, expectation, random permutations, and elementary bounds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- Distinguish input-distribution assumptions from internal algorithmic randomness.
- Use indicators and linearity without unjustified independence.
- Analyze uniform permutations and randomized expected costs.
- Derive expectations and data-structure costs from random splits and comparison indicators.

Sections 5.1–5.4 cover hiring, indicators, randomized algorithms, and further probabilistic applications.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.1 The hiring problem

The n candidates have distinct comparable ranks, with larger better. After each interview, replace the current employee if the candidate is better. Initialize with a dummy worse than every candidate. The quantity of interest includes replacement cost, not just comparisons.

```
HIRE(A):
    best = dummy; hires = 0
    for i = 1 to n:
        interview A[i]
        if rank(A[i]) > rank(best):
            best = A[i]; hire best; hires += 1
    return (best, hires)
```

After round i, best is the greatest of the i interviewed candidates, immediately by induction. With interview fee c_i per candidate and hiring fee c_h per replacement, cost is nc_i+Xc_h, where X counts hires. At n=0 return the dummy with X=0; at $n \geq 1$, $1 \leq X \leq n$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Worst inputs, input distributions, and random bits

Ranks [1,2,…,n] set a record every time, so X=n. Ranks [n,n−1,…,1] give X=1. Deterministic HIRE has a fixed cost on a fixed input.

Average-case analysis needs a distribution. Initially assume all n! orders of distinct ranks are equally likely, a joint condition stronger than equal position marginals. An internally uniform shuffle converts every fixed input into this distribution. Expectations then concern internal randomness instead of an unknown input distribution.

This does not eliminate bad executions: shuffling can still produce increasing order. Randomization guarantees expected cost for each fixed input, not small worst-case cost for every random outcome.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.2 Indicator random variables

For an event E, indicator I_E is 1 on E and 0 otherwise, so $E[I_E] = P(E)$. For $X = \sum_i X_i$, exchanging finite sums on a finite sample space gives

$$E[X] = \sum_{\omega} P(\omega) \sum_{i} X_i(\omega) = \sum_{i} \sum_{\omega} P(\omega) X_i(\omega) = \sum_{i} E[X_i].$$

This is linearity of expectation, requiring no independence. Independence may be needed to calculate individual probabilities or factor intersections. If $X_1=X_2$ indicate heads on the same fair coin, $E[X_1+X_2]=1$ still holds, but $P(X_1=X_2=1)=1/2$ rather than $1/4$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The harmonic hiring expectation

Let X_i indicate hiring candidate i. Conditional on the set of the first i candidates, their i! relative orders are uniform. Their greatest candidate occupies position i with probability 1/i, exactly the condition $X_i=1$. Therefore

$$E[X] = \sum_{i=1}^{n} P(X_i = 1) = \sum_{i=1}^{n} \frac{1}{i} = H_n.$$

No full distribution of X or independence of record events is needed. Integral comparison of 1/x gives $\ln(n+1) \le H_n \le 1 + \ln n$. Thus E[X]=Θ(log n), and expected fees are $nc_i+c_hH_n$; interviews do not become logarithmic.

Ranks [2,5,1,4,6,3] set records at 1,2,5, giving X=3. The uniform expectation at n=6 is $H_6=49/20=2.45$; one trace need not equal it. Exactly one hire occurs iff the best arrives first, with probability 1/n. Hiring everyone requires the sole increasing permutation, with probability 1/n!.

Indicators can count pairs

In a uniform permutation of distinct keys, the inversion indicator for i<j has probability 1/2: swapping those positions bijects the two relative orders. Thus $E[V] = \binom{n}{2}/2 = n(n-1)/4$. Shared positions make indicators dependent but do not invalidate linearity.

Uniformly return n distinct hats bijectively to n owners. Each owner gets the correct hat with probability 1/n, so expected fixed points equal 1 for n≥1. This does not ensure one match in every outcome: at n=2 there are either zero or two, each with probability 1/2. At n=0 there are zero, and 1/n is undefined.

First prove that the indicator sum equals the target count, then evaluate individual probabilities. Addition of disjoint-event probabilities differs from addition of expectations of dependent variables.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.3 Randomized algorithms

RANDOM(a,b) returns every integer in the inclusive range with probability $1/(b-a+1)$, independently across calls. A finite-state pseudorandom generator approximates this probability model; uniformity and independence remain explicit assumptions.

From independent fair bits, use rejection sampling for $m=b-a+1$ outputs. If $m=1$ return a. Otherwise draw $k=\text{ceil}(\lg m)$ bits as z in $0..2^k-1$. If $z<m$ return $a+z$; otherwise redraw the whole group. Acceptance probability $m/2^k$ exceeds 1/2, and accepted legal z values are equiprobable. Expected rounds are less than 2 and expected bit cost $O(\log m)$.

Using z mod m is biased unless m divides 2^k. For three bits modulo 3, residues 0 and 1 have three preimages each, but residue 2 has two.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An in-place uniform shuffle and its joint invariant

```
SHUFFLE(A):
    for i = 1 to n:
        j = RANDOM(i,n)
        swap A[i],A[j]
```

Give elements distinct identities even when their keys agree. Before round i, any specified ordered sequence of i−1 distinct identities is the prefix with probability (n−i+1)!/n!. The initial empty sequence has probability 1. Conditional on that prefix, each remaining identity occupies one suffix position, and RANDOM selects the specified next identity with probability 1/(n−i+1).

The specified i-prefix therefore has probability (n−i)!/n!, preserving the invariant. At termination each complete permutation has probability 1/n!, proving joint uniformity. At n=0 there is one empty permutation and no iterations. Unit-cost random selection gives Θ(n) time and O(1) extra space; a bit-based implementation needs its separate random-bit cost.

Every round of one shuffle

Start A=[A,B,C,D] and choose j=3,4,3,4.

i	Available indices	j	Array after swap	Fixed prefix
1	1..4	3	[C,B,A,D]	C
2	2..4	4	[C,D,A,B]	C,D
3	3..4	3	[C,D,A,B]	C,D,A
4	4	4	[C,D,A,B]	C,D,A,B

This path has probability (1/4)(1/3)(1/2)・1=1/24. Every target permutation has exactly one legal j sequence: find its next identity in the remaining suffix, swap, and continue. Thus all 4!=24 paths are equiprobable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every round of one shuffle (continued)

Start [A,B,C,D]: select only from the unfixed suffix

	Pos 1	Pos 2	Pos 3	Pos 4	Round probability
i=1, j=3	C	B	A	D	1/4
i=2, j=4	C	D	A	B	1/3
i=3, j=3	C	D	A	B	1/2
i=4, j=4	C	D	A	B	1

Green cells stay fixed. This path has probability $1/4 \cdot 1/3 \cdot 1/2 \cdot 1 = 1/24$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Uniform marginals do not suffice

A uniformly chosen cyclic shift puts every identity at each position with probability 1/n, yet for $n \geq 3$ has only n possible outputs rather than n!. The joint distribution is not uniform.

Another algorithm swaps each i=1..n with j chosen from all 1..n. At n=3 it has $3^3=27$ equiprobable paths. Uniformity among six permutations would require 27/6 paths per output, not an integer. Later swaps can disturb already processed positions, invalidating the fixed-prefix invariant.

Stopping the correct shuffle after m rounds gives each ordered m-sample probability $(n-m)!/n!$. Each unordered subset has m! orders, hence probability $1/\binom{n}{m}$. This assumes random access to the suffix; any initial construction of an n-element array must still be charged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Random priorities and collisions

Give each identity an independent uniform priority in {1,…,M}, then sort by priority. Conditional on all priorities being distinct, label symmetry makes all n! orders uniform. Comparison sorting takes Θ(n log n) and storing priorities O(n) space.

A pair collides with probability 1/M. The union bound gives $P(\text{some collision}) \le \binom{n}{2}/M$. For $M=n^3, n \ge 2$, this is at most $(n-1)/(2n^2) < 1/(2n)$: rare, not impossible.

Stable tie breaking leaves unconditional bias. At n=2, the first element remains first with probability $(M+1)/(2M) > 1/2$. Redrawing the entire group after any collision restores uniformity through label symmetry of acceptance; at this M the expected number of rounds is bounded. Uniformly shuffling each tie group also works. The in-place SHUFFLE avoids the collision condition from the start.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Randomization gives a guarantee for each fixed input

RANDOMIZED-HIRE shuffles, then runs HIRE. Every fixed distinct-rank input becomes uniform, giving $E[X]=H_n$ and expected fees $nc_i+c_hH_n$. Shuffling adds $\Theta(n)$ operations, measured separately from hiring fees.

When candidates must be interviewed in external arrival order and cannot be rearranged, this shuffle is unavailable. The H_n result then still needs an arrival-distribution assumption; internal randomness cannot silently control external arrivals.

Shuffling and random-pivot sorting preserve answer correctness on every random path; only cost varies. Such algorithms are generally called Las Vegas. Monte Carlo algorithms allow errors and need separate error probabilities and quantifiers. Expected time alone supplies no correctness probability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.4 Probabilistic analysis and further uses of indicator random variables

Birthday models count colliding pairs or collision existence; balls and bins count loads or waiting times; streak models count windows or longest length; online selection measures the probability of choosing the best. Expectations, probabilities, and maxima are different quantities.

Three tools follow directly. A union's indicator is at most the sum of event indicators, giving the union bound. For nonnegative X, $X \geq tI(X \geq t)$, giving Markov's inequality $P(X \geq t) \leq E[X]/t$. For nonnegative integer L, $L = \sum_{j \geq 1} I(L \geq j)$, giving $E[L] = \sum_{j \geq 1} P(L \geq j)$. Finite sums can be exchanged directly; infinite geometric waiting sums use monotone limits of nonnegative partial sums.

Whenever independence is needed, identify the independent trials. Overlapping adjacent windows cannot be multiplied as if independent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exact birthday collision probability

Let k people draw independently and uniformly from n≥1 dates. For k≤n, conditional on the first j being distinct, person j+1 has n−j unused dates. Hence

$$P(\text{all distinct}) = \prod_{j=0}^{k-1}(1 - j/n), \qquad P(\text{collision}) = 1 - \prod_{j=0}^{k-1}(1 - j/n).$$

At k=0,1 collisions are impossible. For k>n the pigeonhole principle gives probability 1; do not extend the product into negative factors. For 2≤k≤n, 1−x≤e^(−x) bounds the no-collision probability by exp(−k(k−1)/(2n)). Thus k(k−1)≥2n ln 2 is sufficient for collision probability at least 1/2.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Expected pairs differ from an event probability

Let I_{ij} indicate equal dates. Independence and uniformity give $P(I_{ij} = 1) = \sum_r (1/n)^2 = 1/n$, hence $E[X] = \binom{k}{2}/n$ colliding pairs. Independence among these indicators is unnecessary.

A collision means $X \geq 1$. Markov gives only $P(X \geq 1) \leq E[X]$. A large expectation alone gives no probability lower bound: $X = M^2$ with probability $1/M$ and zero otherwise has expectation M but a small chance of being positive.

For independent uniform dates, use both the exact complement and expectation. If $k = o(\sqrt{n})$, the union bound tends to zero. For $k \geq C\sqrt{n}$ with sufficiently large C, the exponential complement bound gives a fixed positive collision probability. These bounds establish the $\sqrt{n}$ threshold scale.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Sixteen dates: exact numbers

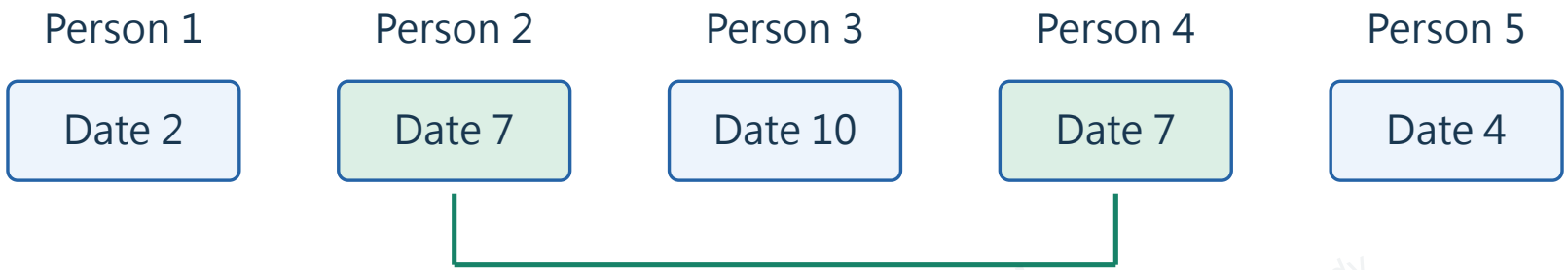
For $n=16, k=5$, no-collision probability is $15 \cdot 14 \cdot 13 \cdot 12/16^4 = 4095/8192$. Collision probability is $4097/8192 \approx .500122$, while expected colliding pairs equal $10/16 = 5/8$. They answer different questions.

At $k=5$ the exponential upper bound on the complement is $e^{(-10/16)} > 1/2$, so it does not prove probability above one half. At $k=6$ it gives $e^{(-15/16)} < 1/2$. A sufficient bound need not locate the exact threshold.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sixteen dates: exact numbers (continued)

One date sample and its model: distinguish counts from expectations



Only colliding pair in this sample: (2,4)

Model: 16 dates, 5 people. P(collision)=4097/8192≈0.500122.

E[colliding pairs]=10/16=5/8, distinct from the event probability above.

Uniform marginals alone could make every date the same random Z, giving collision probability 1, not pair probability 1/n. Pairwise independence suffices for the pair expectation but not the full no-collision product.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Pairwise versus mutual independence

Let U,V be independent uniform values in $\{0,1,2\}$; dates are $B_1=U, B_2=V, B_3=2U-V \bmod 3$. Any pair uniquely determines U,V, so each pair is uniform over nine ordered choices and is independent.

When $U=V$, all three dates agree, giving three cases. When $U \neq V$, all dates differ, giving six cases. Collision probability is therefore $1/3$, unlike $7/9$ for three mutually independent dates. Both models have expected colliding pairs $3 \cdot (1/3)=1$. The first produces three pairs with probability $1/3$; the second has another distribution.

Pairwise independence suffices for this expectation. The complement product requires stronger joint information. Equal expectations do not determine equal distributions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Loads, empty bins, and singleton bins

Throw t balls independently and uniformly into b≥1 bins. A fixed bin's load N sums t hit indicators, so E[N]=t/b and

$$P(N = r) = \binom{t}{r}(1/b)^r(1 - 1/b)^{t-r}.$$

Let Z_0 count empty bins and Z_1 singleton bins. Then $E[Z_0]=b(1-1/b)^t$ and $E[Z_1]=t(1-1/b)^{(t-1)}$, the latter for t≥1. At t=0, $Z_0=b, Z_1=0$. For b=1 use the direct event or the finite-product convention $0^0=1$.

At t=b=4, $E[N]=1, E[Z_0]=81/64, E[Z_1]=27/16$. Loads are dependent because their sum is fixed at t. Average load one does not mean one ball per bin.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Geometric waits and collecting every bin

A fixed bin is hit with probability 1/b on each independent throw. Its waiting time satisfies P(T>t)=(1−1/b)^t, so E[T]=$\Sigma_{t\geq 0}$P(T>t)=b.

After seeing i different bins, a new bin is hit with probability (b−i)/b. The next stage has mean b/(b−i). Regardless of dependence between stage durations, linearity gives

$$E[C] = \sum_{i=0}^{b-1} E[T_i] = b \sum_{j=1}^{b} 1/j = bH_b = \Theta(b \ln b) \quad (b \geq 2).$$

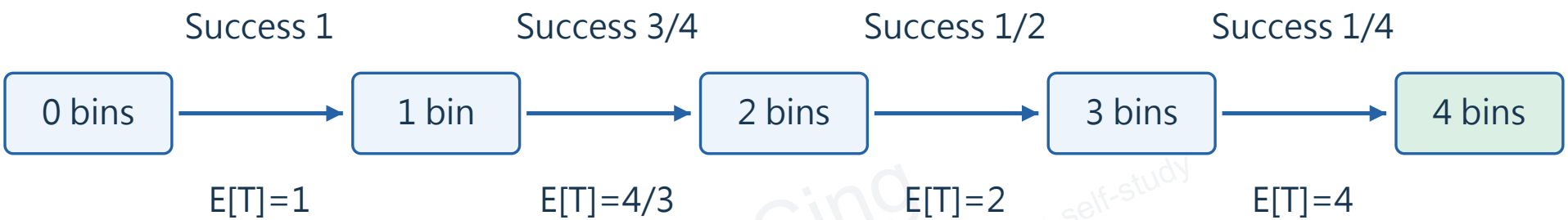
At b=1,C=1. For b=4 the stage expectations are 1,4/3,2,4, totaling 25/3. This is an expected waiting time, not a deterministic completion at a fractional ball count.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Geometric waits and collecting every bin (continued)

Collecting four bins: a repeated hit leaves the state unchanged

State = distinct bins seen. Advance only on a new bin.



Sample throws: 2,2,4,1,4,3. New bins appear at throws 1,3,4,6.
Stage means sum to 1+4/3+2+4=25/3, not a fixed stopping time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Coverage and first collision have different scales

After t throws, a specified bin is empty with probability $(1-1/b)^t \le e^{(-t/b)}$. The union bound gives $P(C > t) \le be^{-t/b}$. Thus $t \ge b(\ln b + c)$ makes failure probability at most $e^{(-c)}$, without independent empty-bin events.

For first repeated-bin time D, P(D>k) is exactly the birthday no-collision event, at most $\exp(-k(k-1)/(2b))$. For $k \ge 2$, $k(k-1) \ge k^2/2$. The tail sum gives

$$E[D] \le 2 + \sum_{k \ge 2} e^{-k^2/(4b)} \le 2 + \int_0^\infty e^{-x^2/(4b)} dx = 2 + \sqrt{\pi b}.$$

For $k \le \sqrt{b}/2$, the union bound makes collision probability at most $k(k-1)/(2b) \le 1/8$. The first $\Theta(\sqrt{b})$ tails are at least 7/8, giving $\Omega(\sqrt{b})$.

First collision therefore has mean $\Theta(\sqrt{b})$, whereas collecting all bins has mean $\Theta(b \log b)$. Different stopping conditions produce different scales.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Streak windows and longest length

Flip n independent fair coins. Let L be the longest H run, zero if no H occurs. For $1 \le k \le n$, let A_{ik} mean all H in the k-window starting at i, with $1 \le i \le n-k+1$. Its probability is $2^{(-k)}$, so the expected number of such windows is $(n-k+1)/2^k$.

Overlapping windows are dependent: $A_{1,3}$ and $A_{2,3}$ both require four initial Hs, probability 1/16 rather than $(1/8)^2$. Expectations still add. The event $L \ge k$ is their union and needs union or joint analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Streak windows and longest length (continued)

Four consecutive Hs contain two dependent length-3 windows



Three length-3 all-H windows start at 1,5,6.

Longest run: 5..8, length 4.

Overlapping windows can both occur; starts 5 and 6 share two flips.

The illustrated sequence has length-3 windows starting at 1,5,6, totaling three. Four consecutive Hs in the second run contribute two overlapping windows, not two disjoint runs. Longest length L=4 differs from window count 3.

General upper and tail bounds for longest streaks

The union bound gives P(L≥k)≤min(1,n2^(−k)). For n≥1 set k_0=ceil(lg n). The integer tail identity gives

$$E[L] = \sum_{k=1}^{n} P(L \geq k) \leq k_0 + \sum_{k=k_0+1}^{\infty} n2^{-k} = k_0 + n2^{-k_0} \leq \lceil \lg n \rceil + 1.$$

For r≥1 and k=ceil(r lg n), the tail is at most n^(1−r). For n=1000,k=20, the sharper window count gives $981/2^{20}<1/1000$; k=30 gives at most $1000/2^{30}<10^{-6}$. Tail guarantees describe individual-run risk beyond the expectation.

At n=0 set L=0 directly instead of taking log 0. For k>n the event is impossible.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A lower bound from independent disjoint blocks

For sufficiently large n let s=floor((lg n)/2)≥1,m=floor(n/s). Partition into m disjoint length-s blocks, ignoring the remainder. Each all-H event has probability 2^(−s)≥1/$\sqrt{}$n. Distinct blocks use disjoint independent coins, making these events mutually independent.

$$P(L < s) \le (1 - 2^{-s})^m \le e^{-m2^{-s}} \le e^{-m/\sqrt{n}} \longrightarrow 0,$$

Here m=Θ(n/log n), so m/$\sqrt{}$n→∞. Consequently E[L]≥sP(L≥s)=Ω(log n), matching the upper bound. Merely having many expected overlapping windows would not prove this lower bound; independent blocks establish high probability of at least one success.

For n=16,s=2,m=8, at least one successful block has probability 1−(3/4)⁸=58975/65536. Hence E[L]≥2・58975/65536, a finite lower bound rather than an exact expectation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete online-selection model

Let $n \geq 2$ distinct-quality candidates arrive in uniform random order. Only seen candidates can be compared; rejected candidates cannot be recalled, and exactly one hire is allowed. Fix an integer $1 \leq k < n$. Reject the first k while recording their maximum M, then select the first later score exceeding M. If none does, select the last candidate.

```
ONLINE-SELECT(n,k):
    M = -infinity
    for i = 1 to k: M = max(M, score(i))
    for i = k+1 to n:
        if score(i) > M: return i
    return n
```

M need not change in the second loop. Before the first exceedance all new scores are at most M; at the first exceedance the procedure returns. Known n, uniform arrivals, distinct scores, and no recall are premises. Arbitrary arrival distributions do not inherit a $1/e$ guarantee.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Success probability by conditioning

Suppose the best candidate is at i. If $i \le k$ the strategy fails. For $i > k$ it succeeds iff the maximum of the first $i-1$ lies among the first k. Otherwise an earlier later-stage record already triggered selection; conversely, when the maximum is in the sample no earlier candidate exceeds M.

The best position has probability 1/n. Conditional on that position, the earlier relative order remains uniform, giving success probability $k/(i-1)$. The i events are disjoint, so

$$P_k = \sum_{i=k+1}^{n} \frac{1}{n} \frac{k}{i-1} = \frac{k}{n} \sum_{j=k}^{n-1} \frac{1}{j}.$$

Conditioning proves the formula without asserting unproved independence. At n=1 select the sole candidate, with probability 1. Define k=0 separately as selecting the first candidate, probability 1/n; do not multiply zero by an undefined harmonic term.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Finite thresholds and the 1/e limit

Let $q_k = \sum_{j=k}^{n-1} 1/j$. The exact difference is $P_{k+1} - P_k = (q_{k+1} - 1)/n$. Since q decreases strictly, an optimal threshold has $q_k \geq 1, q_{k+1} \leq 1$. Equality can give adjacent tied optima. The finite answer need not equal n/e.

Integral bounds give $\ln(n/k) \leq q_k \leq \ln(n/k) + 1/k$, hence

$$\frac{k}{n} \ln \frac{n}{k} \leq P_k \leq \frac{k}{n} \ln \frac{n}{k} + \frac{1}{n}.$$

For x=k/n, f(x)=x ln(1/x) has derivative ln(1/x)−1 and second derivative $-1/x<0$. Its maximum is at x=1/e. Integer k near n/e gives f(k/n)=1/e−O(1/n), so success is at least 1/e−O(1/n). The optimum is at most 1/e+1/n. Therefore the limit is 1/e, without an extra 1/k factor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Six-candidate thresholds and trace

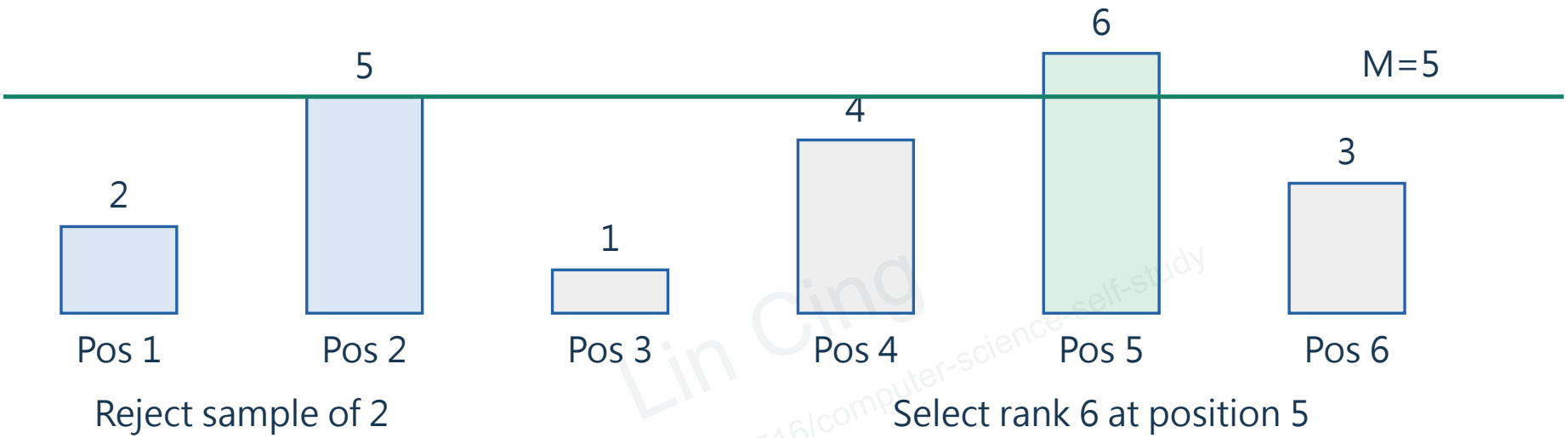
For n=6, probabilities for k=1,2,3,4,5 are respectively 137/360,77/180,47/120,3/10,1/6. Since $q_2=77/60>1$ and $q_3=47/60<1$, k=2 is optimal.

On ranks [2,5,1,4,6,3], rejecting the first two gives M=5. Scores 1 and 4 at positions 3 and 4 fail to exceed it. Score 6 at position 5 triggers selection; position 6 need not be interviewed. This trace chooses the global best, though other permutations fail.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Six-candidate thresholds and trace (continued)

Ranks [2,5,1,4,6,3]: sample maximum 5; first exceedance at position 5



On [5,6,1,2,3,4], the global best is rejected in the sample. No later score exceeds M=6, so fallback selects 4. Correctly executing the strategy does not guarantee success on every permutation.

Extension: a random recursive split

For size $n \geq 2$, print n stars, choose p uniformly from 1..n−1, and recurse on p and n−p. A size-1 call prints one exclamation mark and stops. Let L_n, S_n be expected exclamation and star counts, with $L_1 = 1, S_1 = 0$.

Condition first on p and add child costs, then average:

$$L_n = \frac{2}{n-1} \sum_{j=1}^{n-1} L_j, \qquad S_n = n + \frac{2}{n-1} \sum_{j=1}^{n-1} S_j.$$

Every split preserves total size, and every leaf has size 1, so every execution tree has exactly n leaves. $L_n = n$ is stronger than an expectation. Height and star count still depend on p; extreme splits can create chains without changing the leaf count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Closed form for random splits

For L_n, strong induction substitutes L_j=j and gives $2\sum_{j<n} j/(n-1) = n$. For stars, multiply by n−1 to obtain $(n-1)S_n = n(n-1) + 2\sum_{j<n} S_j$. Subtract this from the corresponding n+1 equation to get

$$S_{n+1} = 2 + \frac{n+1}{n} S_n.$$

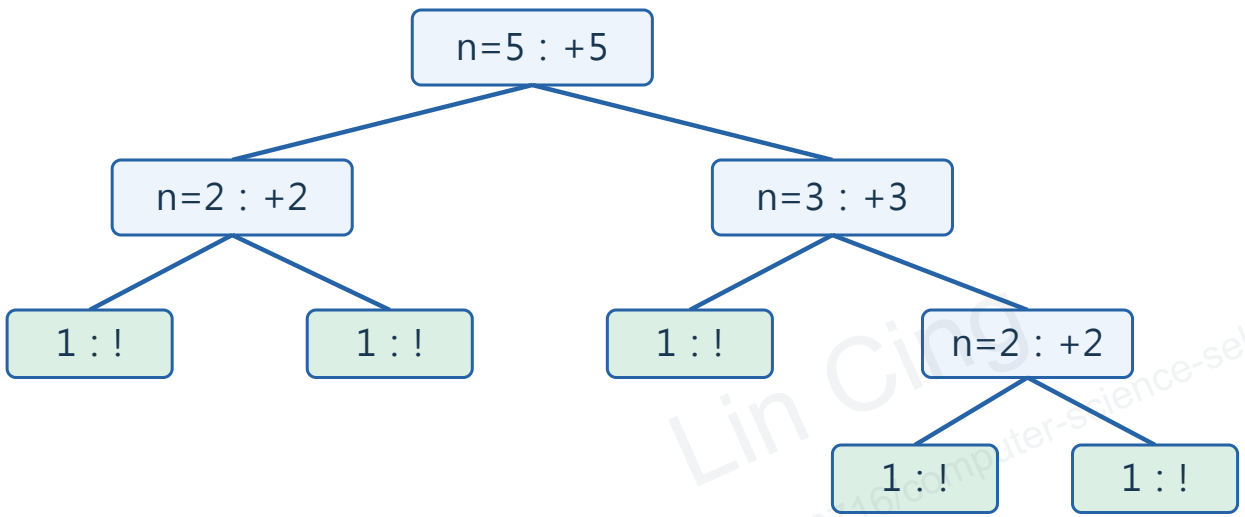
With Q_n=S_n/n, Q_{n+1}=Q_n+2/(n+1). Since Q_1=0, summing gives Q_n=2H_n−2, hence S_n=2n(H_n−1)=Θ(n log n). It is expected cost, not the cost of every path.

n	Leaves in every execution	Expected stars
2	2	2
3	3	5
4	4	26/3
5	5	77/6
6	6	87/5

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Closed form for random splits (continued)

Five unit leaves: leaf count is conserved; star cost varies



Leaves here=5

Stars here=12

E[stars]=77/6

+number: stars at this call

!: one exclamation mark

Shown: p=2, then 3=1+2. All splits give 5 leaves; total stars depend on splits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extension: expected Quicksort comparisons

For distinct keys, independently choose a uniform pivot in every subproblem of size at least 2. Compare each other key once to the pivot, partition into smaller and larger groups, recursively sort, and concatenate. Sizes 0 and 1 return directly. Group ordering and induction prove correctness. Count only key comparisons, separately from copying, allocation, and randomness.

Write sorted keys $z_1<\ldots<z_n$, and let I_{ij} indicate a direct comparison. A pair compares at most once, only when one endpoint becomes pivot and is then removed. Equivalently assign a uniform random priority order to all keys and use each subproblem's earliest key as pivot. Remaining relative orders stay uniform, matching recursive uniform pivot choices. The first pivot among $z_i..z_j$ is therefore uniform in that interval. Outside pivots leave the entire interval together in one child.

If the first interval pivot is an endpoint, the pair compares. An interior pivot permanently separates them. Thus $P(I_{ij}=1)=2/(j-i+1)$, and total $X=\Sigma_{i<j}I_{ij}$.

Quicksort sums, numbers, and boundaries

Group by distance d=j−i, with n−d pairs of that distance.

$$E[X] = 2\sum_{d=1}^{n-1} \frac{n-d}{d+1} = 2\sum_{r=2}^{n} \left(\frac{n+1}{r} - 1 \right) = 2(n+1)H_n - 4n.$$

At n=5 this is 2(4/2+3/3+2/4+1/5)=37/5. For n=2,3,4,6, values are 1,8/3,29/6,103/10. Constant per-element partition work gives time Θ(n+X), hence expected Θ(n log n). Extreme pivot sequences still require n(n−1)/2 comparisons. A high-probability bound needs a separate proof and does not follow automatically from expectation.

Distinctness or an explicit duplicate-key policy is necessary. Two-way Lomuto partitioning that sends all equal keys to one side stays quadratic on all-equal input even with random pivots. Chapters 7 and 9 address three-way equal regions. The distinct-key pair probability cannot be transferred unchanged.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 5, §§ 5.1–5.4 and § 7.1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study